

# Six Opaque Blobs to a Rendered Frame: An End-to-End Forensic and Systems Case Study in Executing an Unmodified Windows AAA Title on Linux Across Two GPU Vendors

Shawn  
Luminos OS Project  
Independent Systems Research  
Arch Linux / ASUS ROG Zephyrus G14 (GA403UU)

*Abstract*—We present a complete, measurement-driven account of taking a 37 GB set of six opaque binary containers of unknown provenance and ending with a modern Direct3D 12 title rendering at native resolution on Linux, on both an integrated AMD GPU and a discrete NVIDIA GPU. The work spans four normally disjoint disciplines: binary forensics (establishing that the payload was neither malicious nor protected by a commercial anti-tamper system), archive reverse-engineering (extracting and reassembling a container format whose own installer refused to run), Windows-on-Linux compatibility engineering (a namespace-fallback defect in Wine, a fixed-path configuration lookup, and a divide-by-zero in an Optimus display path), and graphics-driver fault isolation (a segmentation fault inside a proprietary SPIR-V compiler, reached through an assertion that surfaced as nine unpaintable modal dialogs).

Four findings generalise beyond the specific title. First, code-signing verification is a *sufficient* argument about anti-tamper presence: if a retail Authenticode signature still validates over the current bytes, no protection can have been removed, so the protection question is answered without any dynamic analysis. Second, a D3D12-to-Vulkan translation layer does not emit one program per shader; on this workload 4047 of 4430 modules had identical content hashes but different SPIR-V bytes across vendors, with an NVIDIA-only extension present in 4039 modules and in zero of the working card’s modules. Third, an entire class of “hang” is neither a hang nor a crash but a modal dialog that no compositor ever paints, and it is diagnosable by counting Win32 window classes rather than by reading logs. Fourth, and most costly: two days of sophisticated elimination—driver upgrades, cache purges, six Vulkan extensions disabled, a full shader dump-and-diff, eight code-generation flags—all returned the identical failure, while the actual fix was to notice that exactly one translation layer had ever been installed and to substitute a 5 MB library. We report the resulting failure taxonomy, the negative results in full, and an argument for treating unexamined constants as the first bisection axis rather than the last.

*Index Terms*—binary forensics, code signing, entropy analysis, Wine, Proton, Direct3D 12, Vulkan, SPIR-V, vkd3d, GPU drivers, hybrid graphics, differential diagnosis, software preservation, silent failure

## I. INTRODUCTION

### A. The problem, stated plainly

A set of six binary containers totalling 37 GB arrived on a workstation from a third party’s machine. They were said to contain a 2025-era AAA video game. Three questions had to be answered, in order, before a single frame could be rendered:

- 1) **Is the payload hostile?** The provenance was a friend’s Windows PC, and the working hypothesis on arrival was that the files had come from an impersonation site rather than the genuine distributor. If any component were malicious, the correct action was deletion, not analysis.
- 2) **Is the payload protected?** Modern commercial anti-tamper systems insert a virtualising layer between the operating system and the program’s own control flow. Such a layer, or the countermeasures against it, can be categorically incompatible with a compatibility layer such as Wine [8], in which case no amount of engineering effort would ever produce a running program.
- 3) **Can it be made to run?** The containers refused to open under their own installer, and the title targets Direct3D 12 with Shader Model 6.8, ray tracing, and enhanced barriers—the upper bound of what a translation layer can currently express.

Each question turned out to have an answer that was cheaper and more decisive than expected, and each was initially attacked with an unnecessarily expensive method. That gap—between the method first reached for and the method that actually settled the question—is the recurring subject of this paper.

### B. Why this is worth writing down

Case studies of this shape are usually either forensic (“here is the malware we found”) or compatibility-focused (“here is the Wine bug we fixed”). They are rarely both, and the interesting results here live precisely at the seams:

- The malware question and the anti-tamper question were *settled by the same single measurement*—an Authenticode signature validation—because a valid retail signature simultaneously proves nobody added code and proves nobody removed any (Section IV).
- The extraction failure that consumed two working sessions was not a compression, codec, or archive-integrity problem at all. It was a configuration file that a library looked for at one hard-coded absolute path, and the “unsupported compression method” error it produced was technically accurate and completely misleading (Section V).
- The final graphics defect presented as neither a crash nor a hang. Nine threads were blocked inside modal dialogs that the window system created and never painted. The program was, in a strict sense, waiting for the user to click a button that had never been drawn on any display (Section IX).
- The fix, once found, was to replace a single 5 MB shared library. It was found only after everything sophisticated had failed, because the component in question had been an unexamined constant throughout (Section XI).

### C. Contributions

- 1) **A signature-first argument for anti-tamper triage.** We show that Authenticode validation over current bytes is a sufficient and inexpensive proof of anti-tamper absence, and that entropy profiling, section-table analysis, and import-table analysis serve as independent corroboration rather than as the primary instrument (Section IV).
- 2) **A characterisation of vendor-divergent shader translation.** We report, to our knowledge for the first time in the literature, a quantitative diff of the SPIR-V emitted by a single D3D12-to-Vulkan translator for the same title on two GPU vendors: 4047 of 4430 modules share a content hash but differ in bytes, and one vendor-specific SPIR-V extension appears in 4039 modules on one card and none on the other (Section X).
- 3) **A diagnostic technique for invisible modal deadlock.** Counting windows of Win32 class #32770 through the compatibility layer’s own debugger converts an unfalsifiable “it hangs” into a scalar that a bisection harness can optimise (Sections IX, IX-B).
- 4) **A full negative-results table.** Twenty-three distinct hypotheses were tested and eliminated by measurement. We publish all of them, because the value of this work to a future reader is mostly in what they should *not* spend an evening on (Sections X, XI).
- 5) **A silent-failure taxonomy** drawn from the eleven distinct instances encountered here in which a tool reported success, or reported an accurate but misdirecting error, while doing nothing useful (Section XII).

### D. Scope, ethics, and what was not done

This work was performed on hardware owned by the author, on files already in the author’s possession, for two purposes:

determining whether those files were dangerous, and achieving interoperability with an operating system the vendor does not support.

Three boundaries are worth stating explicitly, because a reader could otherwise infer more than was done.

**No protection was circumvented by this work.** The game executable was never modified; the central forensic finding is precisely that it is *byte-identical* to the signed retail build (Section IV). The Steamworks emulation component present in the archive was pre-existing, was authored by a third party, and is *characterised* here—its file sizes, its loader hook, its configuration—but was neither written, improved, nor analysed for the purpose of extending it. Nothing in this paper constitutes a method for defeating a protection system.

**The contrasting title was abandoned, not attacked.** A second title on the same machine does carry a commercial anti-tamper system, and its accompanying countermeasure is a hypervisor that loads beneath the Windows kernel. We report this only to explain why that title is categorically out of scope for a compatibility layer: Wine implements no Windows kernel into which a driver can load, and on Linux the hardware virtualisation extensions are already owned by the host. That is an architectural impossibility argument, and it terminated the investigation rather than directing it.

**The engineering contribution is entirely on the Linux side.** Every fix described in Sections V through XI—the namespace patch, the configuration path, the virtual desktop, the EGL vendor list, the translation-layer substitution—modifies open-source components or their configuration. None of them touches the game.

### E. Roadmap

Section II describes the hardware and software under test, which matters more than usual here because a hybrid-graphics laptop is the source of at least three of the defects. Sections III and IV answer the provenance and protection questions. Section V covers the two extraction failures and the standalone harness built to isolate them. Section VI describes reconstructing a 54 GB installation without its installer, and the a-priori verification technique that made every step checkable in advance. Sections VII through XI are the graphics bring-up: reaching a swapchain, a three-session misdiagnosis of a “dead” Vulkan driver, the invisible modal deadlock, the vendor shader divergence, and the fix. Section XII generalises. Section XIII is an artifact and reproduction index.

Throughout, a claim is only stated as established if a specific measurement supports it, and retracted claims are marked as such rather than deleted—three of the working hypotheses in this investigation were confidently wrong, and the manner of their failure is part of the result.

## II. SYSTEM UNDER TEST

The platform is load-bearing for this paper. A hybrid-graphics laptop with a high-DPI panel under a scaling Wayland compositor supplies, by itself, the root causes of a divide-by-zero in the display path (Section VII), a three-session misdiagnosis of the discrete GPU (Section VIII), a window that

appears absent from screenshots while existing (Section IX-B), and a game that renders in a box covering 40% of the panel (Section XI). None of those would occur on a desktop with one GPU.

#### A. Hardware

TABLE I  
HARDWARE UNDER TEST

| Component         | Detail                                                                                             |
|-------------------|----------------------------------------------------------------------------------------------------|
| Chassis           | ASUS ROG Zephyrus G14, model GA403UU                                                               |
| Integrated GPU    | AMD Radeon 780M (RDNA 3), no dedicated VRAM; borrows from system memory via the GTT                |
| Discrete GPU      | NVIDIA GeForce RTX 4050 Laptop, 6141 MiB dedicated VRAM, PCIe-attached, <i>no attached display</i> |
| Graphics topology | NVIDIA Optimus (hybrid); the discrete GPU renders and copies, it never scans out                   |
| System memory     | 15.6 GiB usable, shared bandwidth 102.4 GB/s with the iGPU                                         |
| Panel             | 2880×1800 physical                                                                                 |
| Storage           | NVMe; game target on an NTFS volume with 699 GB free                                               |

Two hardware facts recur throughout and are worth isolating now.

**The discrete GPU has no monitor attached.** On an Optimus system the panel is wired to the integrated GPU. The discrete GPU is a compute-and-render device whose output is blitted across PCIe. Any code path that enumerates display modes for the rendering adapter therefore encounters an empty set, and code that divides by a refresh rate obtained from that set faults (Section VII).

**The discrete GPU’s memory is dedicated and non-elastic.** Unlike the Radeon 780M, which borrows from system RAM and can therefore be over-committed gracefully, the RTX 4050’s 6141 MiB is a hard budget. Anything already resident on the card is subtracted from what the game can have. During this work a local language-model service held 3576 MiB of it, which is 58% of the card. This did not cause the central defect—we tested that explicitly and report the negative result in Table XIX—but it shapes the final launcher design in Section XI.

#### B. Software

TABLE II  
SOFTWARE STACK

| Layer                  | Version / detail                                 |
|------------------------|--------------------------------------------------|
| OS                     | Arch Linux, rolling                              |
| Display                | KDE Plasma on Wayland, KWin; scale factor 2      |
| X11 clients            | XWayland, <i>unscaled</i>                        |
| NVIDIA driver          | 595.71.05, upgraded to 610.57.04 during the work |
| Mesa                   | 26.1.6 (RADV for the Radeon 780M)                |
| Compatibility layer    | GE-Proton 10-34; system Wine 11.14 for contrast  |
| D3D12 translation      | vkD3d-proton (bundled), 2.14.1, and 3.0.1        |
| D3D11/10/9 translation | DXVK                                             |
| Vulkan loader          | ICD selection via VK_ICD_FILENAMES               |
| EGL dispatch           | libglvnd, vendor list pinned system-wide         |

#### C. The scaling discontinuity

KDE runs at scale factor 2, so the 2880×1800 panel presents as 1440×900 logical pixels to Wayland-native clients. **XWayland is unscaled**, so X11 clients—which includes everything under Wine—work in physical pixels and see a 2880×1800 display.

This produces two distinct and independently confusing artefacts:

- 1) **Screenshots and compositor coordinates are different number systems.** A screenshot is 2880×1800; the compositor’s own cursor and geometry queries answer in 1440×900. Every pixel measured off a screenshot must be halved before it can be used to aim anything. Nothing warns about this; the symptom is aiming consistently twice as far right and down as intended.
- 2) **A correct window looks half-size in compositor reports.** During the deadlock investigation, KWin described the game window as 960×568, which reads as a scaling bug. It is correct: a 1920×1080 physical window is 960×540 logical, plus a 28-pixel titlebar. The window was the right size the entire time.

#### D. The discrete-GPU access gate

The system enforces a default-deny policy on the discrete GPU. The device node `/dev/nvidia0` is owned `root:dgpu` with mode `0660`, and the `dgpu` group is deliberately empty. Access is granted only through a `setgid` helper, `dgpu-exec-v2`, mode `-rwxr-sr-x root dgpu`. The policy exists so that the discrete GPU can reach its runtime power-management sleep state instead of being woken by incidental enumeration.

Two consequences matter for the investigation.

First, **the gate is the intuitive suspect for every discrete-GPU failure and was never once the cause.** We record this because it was the first hypothesis on three separate occasions. It is falsifiable cheaply and in a way worth reusing: the failures reproduce identically *as root* and with the gate open, which removes permissions from the space of explanations. In the central defect the symptom shape also rules it out on its own—a gate failure produces zero dialogs and a silent fallback to the integrated GPU, whereas reaching a Vulkan pipeline-creation call at all proves the device was already opened successfully.

Second, **a `setgid` process is non-dumpable.** Anything launched through the helper has `/proc/PID/mem`, `/proc/PID/maps`, and `/proc/PID/env` readable only by root, even for the invoking user. `/proc/PID/status` remains readable, which is what made the gate’s operation verifiable at all: the game’s status file reported `Gid: 948 948 948 948`, confirming the `setgid` transition had taken effect.

A related defect found during this work is worth one line, because it is a general hazard in privilege-raising wrappers: an earlier version of the helper raised only the *effective* group ID, and `bash` drops that on startup. Any wrapper invoked through a shell script therefore silently lost the privilege it had just acquired. The launcher must invoke the `v2` helper,

which raises the real GID as well, and it must exec the target binary rather than a shell.

### E. Nomenclature used throughout

TABLE III  
TERMS AND ABBREVIATIONS USED IN THIS PAPER

| Term            | Meaning here                                                                                  |
|-----------------|-----------------------------------------------------------------------------------------------|
| <i>box</i>      | one of the six .bin containers                                                                |
| <i>prefix</i>   | a Wine or Proton emulated Windows filesystem and registry                                     |
| <i>ICD</i>      | Installable Client Driver; a vendor's Vulkan implementation                                   |
| <i>DXIL</i>     | the shader bytecode format Direct3D 12 consumes                                               |
| <i>SPIR-V</i>   | the shader bytecode format Vulkan consumes                                                    |
| <i>assert=n</i> | count of Win32 dialog windows found in the prefix; the failure metric defined in Section IX-B |
| <i>#32770</i>   | the Win32 window class of a dialog box                                                        |

## III. PROVENANCE: IS THE PAYLOAD HOSTILE?

### A. The hypothesis on arrival

The files came from a third party's Windows machine. The stated concern was a virus, and the specific theory was that the third party had downloaded from an impersonation site—a lookalike domain serving a reskinned payload under the name of a known repacking group. That theory has a specific and testable consequence: an impersonated repack would not match the genuine distributor's published checksums, and would likely contain persistence mechanisms absent from the genuine article.

A secondary theory, more sophisticated and more alarming, was raised early: that the repacking process might involve *re-implementing* a commercial anti-tamper system, and that such a re-implementation would be an ideal vehicle for injecting code at runtime. This theory is the reason the anti-tamper question (Section IV) had to be answered even though it initially looked like a pure compatibility concern.

### B. Method

Analysis was static and offline. The full triage comprised six independent lines of evidence.

1) *Checksum provenance*: All six containers were verified against the distributor's published MD5 manifest. All six matched. This does not establish that the distributor is benign—it establishes that the files are the ones the distributor published, which directly falsifies the impersonation-site theory.

2) *Installer script recovery*: The installer is an Inno Setup [20] package, whose compiled script is not visible to strings and is not the first compressed object in the file. The recovery procedure is reusable and is documented here because the obvious approach fails:

The `zlb\xla` magic near the start of the overlay marks the *file data*, not the script. Locate the `SetupLdr` offset table by searching for the 12-byte identifier `rDlPtS`, then read nine little-endian 32-bit words: version, total size, EXE offset, uncompressed EXE size, EXE

CRC, `offset0` (script header), `offset1` (file data), and a table CRC. At `offset0`, skip the 64-byte Inno Setup Setup Data (x.y.z) (u) banner, then read a 4-byte CRC, a 4-byte stored size, and a 1-byte compression flag, followed by chunks of `[crc32(4) + up to 4096 bytes]` until the stored size is satisfied. The concatenated payload is raw LZMA1 whose first five bytes are the properties block.

The decompressed script is the authoritative statement of everything the installer will do to the system, and reading it converts the persistence question from a dynamic-analysis problem into a text-search problem.

3) *Persistence audit*: The recovered script contains exactly one registry write: `AppCompatFlags\Layers` set to `RUNASADMIN`. There are no Run keys, no Winlogon modifications, no service installations, and no scheduled tasks.

An important negative: string searches for `Startup` and `RegSvr32` do hit, and in an automated report those hits would be flagged. They are matches against Inno Setup's own built-in English message table—the strings the installer would display if it were performing such an action—not evidence that it performs one. **A string table is not a call site.** This distinction accounted for the majority of the initial false positives.

4) *Network endpoint enumeration*: Every URL in the compiled script resolves to one of three categories: Microsoft redistributables (Visual C++ runtimes, .NET), NVIDIA PhysX, or the distributor's own domain. The Inno Download Plugin is present and wired up—so the installer genuinely can fetch—but the only things it is instructed to fetch are those redistributables.

5) *The alarming-but-benign component*: Two files, `host.cmd` and `hosts.exe`, are the sort of thing that correctly triggers alarm: a program that edits the system `hosts` file. Reading them settles it. They point roughly sixty lookalike domains at a single address and then *remove* any existing entry that blocks the distributor's real domain. This is an anti-impersonation measure authored by the distributor.

The finding is doubly useful. It is benign, and its presence is positive evidence that the repack is the genuine article—which is exactly the question Section III opened with. **The most alarming artefact in the package turned out to be the strongest disproof of the theory that motivated the investigation.**

6) *Differential comparison against a known-clean package*: Twenty-eight leftover temporary directories were found in the Wine prefix, all dated to a single day, indicating the installer had been run and had failed roughly nine times. Every executable and library in those directories was compared byte-for-byte against the payload of an independently verified clean repack from the same distributor. All matched, including the archive decompressor, the download plugin, and the hosts utility.

Four files differed, and each difference is explicable: three are per-repack configuration (one of which contains the au-

thor’s own installation path, so it *must* differ), and the fourth is a newer build of a compression tool with added codec support.

### C. An anomaly correctly excluded

An earlier pass had flagged a Winlogon shell value set to an empty string—a genuine and serious-looking anomaly. Timestamp comparison excluded it: the modification predates the arrival of these files by twelve days. It is unrelated.

We record this because the temptation, when investigating a suspected compromise, is to attribute every anomaly to the incident under investigation. A system of any age contains unrelated anomalies. **Establishing that an artefact is older than the incident is a complete defence, and it is usually one timestamp away.**

### D. Verdict

TABLE IV  
MALWARE TRIAGE: SIX INDEPENDENT LINES, ALL NEGATIVE

| Line of evidence    | Result                                                                                        |
|---------------------|-----------------------------------------------------------------------------------------------|
| Checksum provenance | All six containers match the distributor’s published manifest; impersonation theory falsified |
| Installer script    | Recovered in full; one registry write, and it is a compatibility flag                         |
| Persistence         | None. No Run keys, services, scheduled tasks, or Winlogon modification                        |
| Network             | All endpoints are Microsoft, NVIDIA, or the distributor                                       |
| hosts tooling       | Anti-impersonation, authored by the distributor; evidence <i>for</i> authenticity             |
| Byte comparison     | Payload identical to an independently verified clean package, modulo per-repack configuration |
| Runtime footprint   | Touches only its own directory, system libraries, and its own application-data folder         |

**The files are clean.** No malware, no persistence, no unexpected network behaviour. The investigation is closed and should not be reopened without new evidence.

The remaining theory—that the repack re-implements an anti-tamper system, and that the re-implementation is the injection vector—is not addressed by any of the above. It is addressed, and destroyed, in the next section, by a single observation that also happens to answer the compatibility question.

## IV. PROTECTION STATUS: ESTABLISHING THE ABSENCE OF COMMERCIAL ANTI-TAMPER

This section answers the question the reader most likely arrived for: *how do you determine, from a binary alone, that a commercial anti-tamper system is not present?* The answer has a strong form and a weak form, and most published methodology uses only the weak form.

### A. Why the question had to be answered first

Commercial anti-tamper systems of the Denuvo class operate by taking selected regions of the program’s control flow and replacing them with an interpreter for a private instruction set, generated per build. The practical consequences for this project were three:

- 1) **Compatibility.** Such a layer interacts closely with operating system internals. A compatibility layer that reimplements those internals may or may not satisfy it, and the failure mode is not usually a clean error message.
- 2) **Effort allocation.** If the protection were present, the ceiling on this project might be architectural rather than engineering—in which case the correct decision is to stop, as was in fact taken for the contrasting title discussed in Section IV-D.
- 3) **Threat model.** The outstanding malware theory from Section III was that the protection had been *reimplemented* by a third party, and that this reimplementation was the injection vector. Proving no such layer exists retires that theory entirely.

The subject of analysis is Retail\007FirstLight.exe, 64,228,744 bytes, MD5 3e0c3e10a9fdb15397f5bc50aeb5e09b, extracted from the fifth container.

### B. The strong argument: signature validation

Windows executables may carry an Authenticode signature [3]: a PKCS#7 / CMS structure [4] embedded in the certificate directory of the PE file [5], containing a cryptographic digest computed over the file’s contents with three regions excluded (the checksum field, the certificate table entry, and the certificate data itself).

Validation proceeds in two independent steps, and the distinction between them is the crux of this section:

- 1) **Chain validation** asks whether the signing certificate descends from a trusted root. This says who signed it.
- 2) **Digest validation** recomputes the Authenticode hash over the *file as it exists now* and compares it to the digest stored inside the signature blob. This says whether the file has changed since it was signed.

For this executable, both hold. The recomputed SHA-256 equals the digest embedded in the PKCS#7 structure. The signer is **IO INTERACTIVE A/S**, via GlobalSign GCC R45 CodeSigning CA 2020, with a signing time of 2026-07-20 15:43:44 UTC.

1) *Why this settles the question:* The reasoning is short enough to state completely:

The signature validates over the current bytes. Therefore the current bytes are the bytes the publisher signed. Therefore **nobody has modified this executable**—not to add anything, and, crucially, **not to remove anything either**. Anti-tamper protection of the class in question is applied to the executable at build time by the publisher and lives inside the executable’s own sections. If it had ever been present in this build, it would still be present now, and it would be visible. It is not visible. Therefore it was never applied to this build.

This is a *sufficient* argument, and it is the strong form. It requires no dynamic analysis, no debugger, no emulator, and no unpacking. It is also self-verifying in a way that heuristics

are not: a forged conclusion would require a hash collision against SHA-256.

It carries a second payload for free. The unresolved malware theory—that the protection was reimplemented and used as an injection vector—requires modification of the executable. The signature forecloses that. **One measurement answers both questions.**

2) *The scope of the claim, stated honestly:* The argument establishes that *this executable* is unmodified and unprotected. It does not establish anything about the other 10,949 files in the package; those were covered separately, by the differential comparison in Section III. It also does not establish that no protection exists anywhere in the product—a separate protected loader could in principle exist as a distinct binary. Section IV-C addresses that by enumerating what the executable actually imports and what else ships beside it.

### C. Corroboration: three independent weak arguments

The strong argument is sufficient, but a single-instrument conclusion is a fragile conclusion. Three further lines were run, each of which would independently have raised suspicion had the protection been present.

1) *Entropy profiling:* Compressed, encrypted, and virtualised code is high-entropy; ordinary compiled code is not. Entropy analysis is the standard first-pass instrument for detecting packing [1], with the well-documented caveat that it produces both false positives and false negatives in the general case [2].

Rather than compute a single figure for the file, which averages away exactly the signal of interest, the `.text` section was profiled in 43 windows of 1 MiB each.

TABLE V  
SHANNON ENTROPY PROFILE OF THE GAME EXECUTABLE

| Measurement                              | Value                         |
|------------------------------------------|-------------------------------|
| Whole file                               | 6.354                         |
| <code>.text</code> , 43 windows of 1 MiB | range 5.906–6.700, mean 6.505 |
| Windows at or above 7.5                  | <b>0</b>                      |
| Expected for a virtualised region        | 7.9 and above                 |

The distribution is tight, unimodal, and entirely consistent with optimised native x86-64. There is no high-entropy island anywhere in the code section. Had a virtualising layer been present, it would appear as a contiguous run of windows well above 7.5, and windowing is precisely what makes it visible: a whole-file figure of 6.354 could in principle conceal a small high-entropy region, but 43 individually reported windows cannot.

2) *Section and import table analysis:* Protected builds are structurally distinctive. They typically carry an extra section with a non-standard name, a truncated or stub import table with the real imports resolved at runtime, and a substantial size increase over the unprotected build. None of these is present. The import table in particular is decisive in its own

TABLE VI  
STRUCTURAL INDICATORS

| Indicator          | Observation                                                                                                                                                               |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sections           | <code>.text</code> , <code>.rdata</code> , <code>.data</code> , <code>.pdata</code> , <code>__RDATA</code> , <code>.rsrc</code> , <code>.reloc</code> — stock MSVC layout |
| Anomalous sections | none                                                                                                                                                                      |
| Imports            | 52 DLLs, 753 symbols; a normal compiler-emitted table                                                                                                                     |
| Runtime            | ucrt and VCRUNTIME140                                                                                                                                                     |
| Protection strings | zero hits for the vendor name, VMProtect, or Themida                                                                                                                      |
| Size profile       | the several-hundred-megabyte bloat characteristic of a protected build is absent                                                                                          |

small way: a protected binary usually cannot afford a full static import table, because the protection needs to mediate library resolution.

3) *What the DRM actually is:* Having established what is absent, the analysis identified what is present. The only protection mechanism in the package is Steamworks licensing, and the countermeasure is a third-party Steam API emulator.

TABLE VII  
THE ACTUAL PROTECTION AND COUNTERMEASURE

| File                         | Identification                                                                                                                        |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>steam_api64.rne</code> | 319,584 B — the genuine publisher-shipped Valve library, renamed                                                                      |
| <code>steam_api64.dll</code> | 1,098,392 B — the third-party replacement configuration; carries the author’s banner, the application ID, and a configuration section |
| <code>steam_emu.ini</code>   |                                                                                                                                       |

The mechanism, characterised but not extended (Section I-D), is a library-loading hook that looks for renamed twins of each library. The architecturally relevant point is that **it replaces only Valve’s own API surface**—a documented interface between the game and a storefront—and does not touch the game’s code. This is consistent with, and independently confirms, the signature finding.

### D. The contrasting case, and why it terminated

A second title on the same machine provides the control that makes the above meaningful: it *does* carry the commercial anti-tamper system, still present and active. Its accompanying countermeasure is not a library replacement but a **hypervisor**: a pair of kernel drivers implementing AMD-V and Intel VT-x support respectively, which load beneath the Windows kernel and answer the protection’s queries from a layer below it. Source for the virtualisation component ships in the archive.

This approach cannot work on Linux under a compatibility layer, for two independent reasons, either of which is sufficient:

- 1) **There is no kernel to load into.** Wine implements Windows APIs in user space; it does not implement a Windows kernel, so there is no object into which a `.sys` driver can be loaded.

- 2) **The hardware is already taken.** The CPU’s virtualisation extensions are owned by the Linux host. A second hypervisor cannot claim them.

We note this explicitly as an *architectural impossibility*, in the narrow sense argued in Section XII: not “this would be difficult” but “this specific mechanism has no counterpart on this platform.” The investigation of that title stopped there, which is the correct outcome and the whole value of asking the protection question before the engineering question.

#### E. Summary of the method

For a reader wishing to reproduce this determination on another binary, the procedure in decreasing order of decisiveness:

- 1) **Validate the Authenticode digest over current bytes.** If it validates, the file is the publisher’s build and the question is answered. Do not skip to entropy; this step is both cheaper and stronger.
- 2) **Profile entropy in windows**, not whole-file. Report the window count above threshold, and state the threshold.
- 3) **Enumerate sections and imports.** A full, compiler-shaped import table is difficult for a protection layer to preserve.
- 4) **Identify what protection is present.** “No Denuvo” is not “no DRM,” and conflating them produces a wrong prediction about runtime behaviour.

For this title the answer is unambiguous: **there is no commercial anti-tamper system, and there never was one in this build.** Nothing was stripped, because there was nothing to strip. The engineering path was therefore open, and the remainder of this paper is about walking it.

### V. EXTRACTION: TWO FAILURES WITH ONE SYMPTOM

#### A. The containers

TABLE VIII  
ARCHIVE INVENTORY, VERIFIED BY THE DECOMPRESSOR’S OWN LIST COMMAND

| Box          | Bytes          | Files         | Packed    | Unpacked        |
|--------------|----------------|---------------|-----------|-----------------|
| f g-01       | 24,992,387,781 | 858           | 23,834 MB | 51,419 MB       |
| f g-02       | 6,037,379,414  | 9,774         | 5,757 MB  | 5,833 MB        |
| f g-03       | 3,591,566,415  | 1             | 3,425 MB  | 3,750 MB        |
| f g-04       | 3,280,408,667  | 218           | 3,128 MB  | 4,011 MB        |
| f g-05       | 902,897,841    | 78            | 861 MB    | 1,393 MB        |
| f g-06       | 525,112        | 21            | 0 MB      | 2 MB            |
| <b>Total</b> | <b>≈37 GB</b>  | <b>10,950</b> |           | <b>≈64.8 GB</b> |

Note the last row of Table VIII. The smallest container, at 525 KB and roughly one part in seventy thousand of the total, blocked the entire project for two working sessions. It contains no game data whatsoever.

A reading trap in the decompressor’s own output is worth recording: the size fields reported by its list command are in *megabytes* for the maximum, and the running counter is a 32-bit signed integer that goes negative past 2 GB. Reading the counter rather than the maximum produces nonsense on any modern archive.

#### B. Failure 1: 0.0% forever

The installer, run under Wine, stopped at 0.0% and stayed there. No error, no timeout, no progress. The naive readings—“it is slow”, “it deadlocked”, “the GUI is broken”—are all wrong.

1) *Root cause:* The repack’s external-compressor plugins are split across architectures: a 64-bit helper process does the work, and a 32-bit plugin loaded inside the installer communicates with it through shared memory. Tracing the relevant calls with Wine’s relay debugging revealed the mismatch:

- The 64-bit helper creates its mapping as `Global\Read_MapFile_CLS-srep00`.
- The 32-bit plugin opens `Read_MapFile_CLS-srep00` — **with no prefix**.

On Windows this works, and it works for a reason specific to the NT object manager [19]. A session’s `\Sessions\N\BaseNamedObjects` directory is a *shadow* directory: a name that misses there falls through to the global `\BaseNamedObjects`. **Wine implements no such fallback.** The unprefixed open therefore misses, and the plugin responds the way a 2000s-era Windows utility responds to an unexpected error—it raises a modal message box reading `Can’t open read file mapping!` and waits.

That message box is never painted. The installer is not hung; it is waiting for a click on a button that exists only in the window manager’s data structures. This is the first of two independent occurrences of this failure shape in this project; the second, in the graphics stack, is Section IX.

2) *Fix:* One byte per helper. Zeroing the leading G of the `"Global\"` string literal truncates the prefix to the empty string, so both sides use the session-local name and the open succeeds. Backups of the originals were retained.

3) *Why patching on disk is insufficient:* This is the part that generalises. Patching the helpers where they live has no effect, because **the installer extracts its own fresh, unpatched copies of every helper into a newly created temporary directory on each run.** A hand-patched copy elsewhere is simply bypassed.

The working approach is a live patcher: a process that polls at 0.2-second intervals and patches new copies as they appear. On launch it caught eight unpatched helpers across two separate temporary directories within seconds.

The general lesson is that *a self-extracting installer is a moving target, and static patching of its payload is only valid if the payload is not re-materialised at runtime.* The way to detect this class of problem is to notice that the patch verifiably applied and verifiably had no effect—otherwise an extremely confusing pair of facts.

#### C. Failure 2: “unsupported compression method”

With the modal deadlock fixed, five of the six containers opened. The sixth—the 525 KB one—failed differently:

```
[file] work\work\build01.bat
ERROR: unsupported compression method
ERROR: archive data corrupted (decompression fails)
```

1) *Differential diagnosis: distinguishing the two failures:* The two failures present almost identically to a user: the process is alive and nothing happens. They are distinguished by a single cheap observation, and recording that observation is what prevented a second week on the first bug.

TABLE IX  
TELLING THE TWO EXTRACTION FAILURES APART

|               | Failure 1 (modal)  | Failure 2 (config)  |
|---------------|--------------------|---------------------|
| CPU           | 0%                 | one core fully busy |
| Child process | helper alive       | <b>none</b>         |
| Bytes written | partial            | <b>zero</b>         |
| Time to fail  | never              | $\approx 2$ s       |
| Root cause    | namespace fallback | missing config file |

2) *The wrong hypothesis, and how long it survived:* The error message is a direct statement that a compression method is not supported, so the hypothesis was that the container used a codec the shipped decompressor did not implement. Considerable effort went into it: an archive scanner was written, the tail directory block was located at two byte offsets, and a brute-force search over LZMA1 parameter space was implemented to recover the per-file method table out of the compressed directory block.

None of it was the cause. The archive was correct the entire time.

3) *Root cause:* The decompression library reads its table of external compressors from a configuration file, and it looks for that file at **exactly one hard-coded absolute location**: the root of the system drive, `c:\arc.ini`. Not its own directory. Not the current working directory. Not the output directory. Not the archive’s directory.

This was established, rather than guessed, by tracing file operations with `WINEDEBUG=+file` from three different working directories. Every run produced exactly one probe, always `CreateFileW L"c:\arc.ini"`, and always status `c0000034` — object name not found.

The file exists, in the toolchain directory. With the table absent, every compression method that depends on an external tool resolves to “unsupported,” and extraction dies on the first file that uses one.

4) *Fix:* One line: copy the configuration file to the root of the emulated system drive. The container that had blocked the project for two sessions opened on the first attempt afterwards, as did every other container.

#### D. Three signals that were available the whole time

Each of the following was observable before any of the wasted effort began, and each independently points away from the codec hypothesis.

- 1) **The list operation succeeded throughout.** The directory block parsed correctly and reported all files. A *working list beside a failing extract points at the method table, not at the archive*. Were the archive corrupt or the format misunderstood, listing would fail first.

- 2) **Zero bytes written, failing in two seconds, is a parse-time failure.** A genuine codec or data fault writes a partial output file first, because it fails partway through a stream. Checking the destination directory—not the log—separates these instantly.
- 3) **“Unsupported compression method” from this class of tool almost always means missing configuration, not a missing codec.** The method table is data, and data can simply be absent.

#### E. Dead ends, published so they are not repeated

TABLE X  
FAILURE 2: HYPOTHESES TESTED AND ELIMINATED

| Hypothesis                          | How it died                                                       |
|-------------------------------------|-------------------------------------------------------------------|
| Corrupt configuration               | Diffed against four copies; all identical                         |
| Destination filesystem              | NTFS versus ext4 made no difference                               |
| Temporary path setting              | Changing it made no difference                                    |
| Regression from the Failure 1 patch | Helpers verified still patched                                    |
| Unknown LZMA1 parameters            | Brute-force search found nothing wrong, because nothing was wrong |
| A Wine defect                       | Two different programs failed on the same file in the same way    |

#### F. The standalone harness

To separate “the archive cannot be read” from “the installer cannot read the archive,” a minimal harness was built that loads the decompression library directly and calls its extraction entry point, bypassing the installer framework entirely.

The build is constrained in an instructive way. The library is 32-bit, and the system’s Wine is a new-WoW64 build with no 32-bit Unix support, so a 32-bit *Winelib* binary cannot be produced at all. What can be produced is a 32-bit *PE*: a freestanding executable with no C runtime and no Windows headers, built with the LLVM toolchain already present.

```
llvm-dlltool -m i386 -k -d kernel32.def -l kernel32.lib
clang --target=i386-pc-windows-msvc -nostdlib \
  -ffreestanding -O1 -c arcx32.c
lld-link /subsystem:console /entry:start \
  /nodefaultlib /machine:x86 arcx32.obj kernel32.lib
```

One implementation detail cost an afternoon and is a general hazard when calling into legacy Windows libraries: **the progress callback must be declared `__stdcall`**. Declared `cdecl`, the stack drifts by 16 bytes per invocation and the program page-faults after a few hundred callbacks—far enough into the run to look like a data-dependent fault rather than a calling-convention error.

## VI. REASSEMBLY WITHOUT THE INSTALLER

Extraction yields roughly 64.8 GB of intermediate files across 10,950 objects. These are not the game. They are compressed, delta-encoded, and obfuscated fragments that the installer’s own scripts would normally reconstitute. With the installer unusable for other reasons, the reconstruction had to be performed by hand.

The result: a working 54 GB installation, built in **37 minutes**, without ever running the installer.



### A. The recipe is inside the archive, in plaintext

The planned approach was to reverse-engineer the reconstruction procedure out of the carved installer script (Section III). That was unnecessary. The 525 KB container—the one that had blocked everything—contains the distributor’s own worklists as plain text.

TABLE XI  
CONTENTS OF THE SMALLEST CONTAINER

| File           | Function                                                                                  | Lines |
|----------------|-------------------------------------------------------------------------------------------|-------|
| fitgirl101.txt | restore the audio assets, one command per file                                            | 9,774 |
| fitgirl102.txt | rebuild the first data chunk                                                              | 98    |
| fitgirl103.txt | rebuild the second data chunk                                                             | 335   |
| w01/w02.bat    | de-obfuscate halves; a wait-for-file handshake                                            | 9     |
| build01-03.bat | <b>nothing of value</b> — fetch an advert list from a URL shortener and delete themselves | 3     |

Two observations follow, and the second is the more useful.

First, **the file that stopped both the installer and the standalone harness for two sessions is a three-line advertisement fetcher.** It contains no build logic at all. The project was blocked on an artefact with no function.

Second, **the carved installer script was still worth having,** but for a different reason than expected: the worklists give the commands, and the script gives the *order* and the *working directory* for each stage, which the worklists omit. Neither source is sufficient alone.

### B. Every tool is a renamed public tool

The toolchain shipped in the archive consists of eleven executables with opaque names. None required reverse-engineering. **Running each with no arguments caused it to print its own banner.**

TABLE XII  
TOOLCHAIN IDENTIFICATION, BY RUNNING EACH BINARY WITH NO ARGUMENTS

| Ships as      | Actually is                                        |
|---------------|----------------------------------------------------|
| fgpack.exe    | OGGRE v0.1.1, an Ogg Vorbis recompressor [17]      |
| fgpack2.exe,  | LZ4 at level 12 [14]; two of the three are byte-   |
| fgpack3.exe,  | identical                                          |
| lz4.exe       |                                                    |
| x.exe         | xdelta3, VCDIFF delta decoder [15]                 |
| xx.exe        | a public XOR utility                               |
| x5.exe        | HDiffPatch hpatchz, older build [16]               |
| x5n.exe       | HDiffPatch hpatchz v4.8.0, with directory patching |
| fart.exe      | find-and-replace; expands path placeholders        |
| run.exe,      | URL fetcher; sleep                                 |
| countdown.exe |                                                    |

The obfuscation here is nominal, not technical. This is worth stating because the instinct on encountering eleven unknown binaries inside a package one already suspects of hostility is to disassemble them. **The cheapest identification step—execute with no arguments and read the banner—identified all eleven in under two minutes,** and each identification was then confirmable against the public tool’s documented command-line grammar.

One tool in the set is worth a note for the forensic reader. The XOR utility exists because the distributor XOR-obfuscates selected payload files, here with a 64-bit key, purely to evade antivirus heuristics on the compressed intermediates. XOR is symmetric, so those steps are trivially reversible and cost nothing.

### C. A-priori verification: the most valuable technique in this project

Each directory-level patch is an HDiffPatch directory diff, and hpatchz offers an information mode that prints the diff’s manifest *before* anything is applied: how many input files it expects, their exact total size in bytes, and the exact size of the output it will produce.

This converts a long pipeline from hopeful to checkable. Every stage could be verified against a number known in advance, so an error would be caught at the stage that caused it rather than at the end.

All five checks landed exactly. The first row is the most informative: the manifest’s expectation of 10,456,549,551 bytes across 9,775 files decomposes precisely into 9,774 restored audio files plus one separate container, to the byte. The 243 and 411 input counts likewise decompose exactly into their constituent file categories with nothing unaccounted for.

**The general principle: prefer a pipeline whose stages publish their own preconditions.** Where such a manifest exists, use it as a checksum you already have rather than validating only the final artefact—which, at 54 GB and 37 minutes, is an expensive place to discover a mistake made in minute three.

### D. The trap that would have silently broken everything

The extraction script sent all six containers to a single destination directory. **That is wrong for two of them.**

Had this gone unnoticed, the 9,774 audio files and their companion container would have landed one directory too high, and the directory patch in stage 3 would have found no inputs. The failure would have appeared at a stage unrelated to its cause. The destinations are recorded in the carved installer script, which is the second reason that artefact remained worth having.

### E. The procedure

Ten stages, in order. Every deletion instruction in the original must be honoured: the directory diffs consume their inputs, and hpatchz deletes the old tree itself when it merges its temporary output back.

- 1) De-obfuscate both halves of the payload (XOR, symmetric).
- 2) Restore 9,774 audio assets, one recompressor invocation each.
- 3) Apply the top-level directory patch, producing two large intermediates.
- 4) De-obfuscate both intermediates.
- 5) Run the chunk-0 worklist.

TABLE XIII  
A-PRIORI VERIFICATION: EVERY STAGE CHECKED AGAINST A MANIFEST READ BEFORE THE STAGE RAN

| Stage                  | Expected (from the manifest)              | Measured                                                                       | Match |
|------------------------|-------------------------------------------|--------------------------------------------------------------------------------|-------|
| Audio restore          | 9,775 input files, 10,456,549,551 B total | 9,774 restored files at 6,523,412,777 B, plus one container at 3,933,136,774 B | yes   |
| Directory patch output | 2 files, 12,091,379,792 B                 | 6,613,152,048 + 5,478,227,744                                                  | yes   |
| Chunk 0 inputs         | 243 files, 19,387,609,117 B               | 243 files, 19,387,609,117 B                                                    | yes   |
| Chunk 1 inputs         | 411 files, 32,085,486,829 B               | 411 files, 32,085,486,829 B                                                    | yes   |
| Final outputs          | —                                         | chunk 0 at 20,689,753,541 B; chunk 1 at 35,798,270,141 B                       | —     |

TABLE XIV  
PER-CONTAINER EXTRACTION DESTINATIONS

| Containers          | Destination                              |
|---------------------|------------------------------------------|
| fg-01, fg-04, fg-05 | the application root                     |
| fg-02, fg-03        | the application's runtime subdirectory   |
| fg-06               | the temporary directory (worklists only) |

- 6) Apply a patch *to the chunk-0 patch*—the directory diff is itself shipped as a diff.
- 7) Apply the chunk-0 directory patch, producing the first data archive.
- 8–10. Repeat stages 5–7 for chunk 1.

The per-file shape inside the chunk worklists is consistent: LZ4 to rebuild the compressed stream, xdelta3 to apply the residual, then XOR, then move into place.

*F. Performance: process startup dominates, not compression*

TABLE XV  
MEASURED WALL-CLOCK TIME PER STAGE

| Stage                                          | Time            |
|------------------------------------------------|-----------------|
| De-obfuscate 10.5 GB                           | 16 s (675 MB/s) |
| 9,774 recompressor invocations, 6-way parallel | 5 min           |
| Top-level directory patch                      | 27 s            |
| Chunk-0 worklist                               | 5 min 35 s      |
| Chunk-0 archive                                | 35 s            |
| Chunk-1 worklist                               | 21 min          |
| Chunk-1 archive                                | 1 min 40 s      |
| <b>Total</b>                                   | <b>≈37 min</b>  |

The dominant cost in the two worklist stages is **Wine process startup, not compression**. Each individual recompressor invocation performs roughly 40 ms of actual work; the remainder is the cost of creating a Windows process under a compatibility layer. Parallelism is therefore the only tuning knob that matters for those stages, and a six-way parallel invocation reduced 9,774 sequential launches to five minutes.

One parameter was deliberately reduced from the distributor's own commands: a cache-size flag was lowered from 4 GB to 1 GB. On a 15.6 GiB machine the larger value risks the userspace out-of-memory killer, and the change cost nothing measurable.

### G. Result

A 54 GB installation with the signature-verified original executable, both reconstructed data archives, the encrypted

package definition the engine expects, and the Steamworks emulation component in place. The source containers were kept read-only throughout, so any stage remained redoable: recovery from a mistake is re-extracting the affected container, roughly 16 minutes for all six.

## VII. FIRST GRAPHICS BRING-UP: A NULL POINTER WITH A DOCUMENTED CAUSE

The installation is complete and the executable is the publisher's own. What follows is the second half of the project, which took considerably longer than the first, and which is entirely about the graphics stack.

### A. Establishing that the reconstruction is actually the game

Before diagnosing rendering, it was worth confirming that the 54 GB tree is a coherent game rather than a coherent-looking pile. Four observations settle it, and all four come from the game's own startup behaviour rather than from our inspection:

- The engine locates and reads its encrypted package definition, `packagedefinition.txt` at 35,284 bytes, which is the file the Glacier engine consults to find every other asset. A wrong or truncated copy stops startup immediately.
- The crash-telemetry component initialises and writes `crash_metrics.dat`, 25,240 bytes.
- NVIDIA's Streamline framework loads seven `sl.*` libraries and *verifies its own digital signatures on each of them*, which is a third-party integrity check passing on our reconstruction, performed by code we did not write and cannot influence.
- Streamline then allocates its device-side and host-side pools, `sl.chi.heapGPU` and `sl.chi.heapCPU`.

The game reaches its rendering path. Everything after this point is a graphics problem, not a packaging problem.

### B. The bundled translation layer is too old

Direct3D 12 is translated to Vulkan by `vk3d-proton` [10]. The system Wine ships a much older `vk3d`, and its failure is immediate and legible:

```
Enhanced barriers: 0
```

The game requires enhanced barriers, a Direct3D 12 feature for expressing resource state transitions, and refuses to proceed without them. This is a clean, early, well-signposted failure—the kind one wants.

Replacing the translation layer with vkd3d-proton 2.14.1 and the Direct3D 11 path with DXVK [11] changes the log to:

```
DX Ultimate supported!
DXR 1.1 support enabled
Enabling support for SM 6.8
```

Ray tracing, mesh shaders, and Shader Model 6.8 all negotiate successfully. The game then crashes.

### C. The crash: a null vtable, disassembled

The fault is a read from address zero, and the three instructions around it say exactly what happened:

```
140cb497c: mov 0x8(%rax),%rcx ; load an interface ptr
140cb4997: mov (%rcx),%rax    ; load its vtable <--
          FAULT
140cb49a8: call *(%rax)        ; call through it
```

This is the canonical shape of an unchecked COM interface query. The game asked for an interface, did not check the return value, stored the null it received, and called a method on it one basic block later.

1) *Which interface:* The requested interface is ID3D12GraphicsCommandList10, IID {7013c015-d161-4b63-a08c-238552dd8acc}.

It is a recent addition to the Direct3D 12 interface family. vkd3d-proton 2.14.1 does not implement it and correctly returns E\_NOINTERFACE; the game ignores that and dereferences the null.

2) *Verifying support without running anything:* A useful and very cheap technique: a COM interface’s presence in a translation layer can be checked by searching the binary for the IID in its *little-endian in-memory byte order*, which is not the order the GUID is printed in.

```
grep -qaF $'\x15\x00\x13\x70\x61\xd1\x63\x4b'\
$'\xa0\x8c\x23\x85\x52\xdd\x8a\xcc' d3d12core.dll
```

The first three fields byte-swap; the last two do not. Getting this wrong produces a confident false negative.

A second false-negative source cost real time and is worth stating plainly: inside a Proton distribution the Direct3D 12 libraries live under files/lib/wine/vkd3d-proton/x86\_64-windows. **That is lib, not lib64.** A lib64 path exists and contains other things, so a search there returns nothing and looks like an answer.

A third: Wine ships its own d3d12.dll of roughly 124 KB, which is a thin forwarding stub and is *not* vkd3d-proton. Confirming the size distinguishes them at a glance.

### D. An unrelated fault in the same session: divide by zero in the display path

Independently of the interface problem, the game faulted while enumerating display modes. The log sequence is self-explanatory once the hardware topology from Section II is in mind:

```
Found monitors not associated with any adapter,
using fallback
readMonitorEdidFromKey: Failed to get EDID
reg key size
```

TABLE XVI  
INSTALLED TRANSLATION-LAYER LIBRARIES, FOR SIZE COMPARISON

| Library       | Bytes     |
|---------------|-----------|
| d3d12.dll     | 159,758   |
| d3d12core.dll | 5,185,550 |
| d3d11.dll     | 6,209,550 |
| d3d10core.dll | 290,830   |
| dxgi.dll      | 4,435,982 |
| nvapi64.dll   | 1,974,286 |
| nvofapi64.dll | 1,384,462 |

The discrete GPU has no attached display. Enumerating modes for the rendering adapter therefore yields an empty set, and the engine divides by a refresh rate taken from that empty set.

The fix is to give the application a display to find: a Wine virtual desktop, at which point mode enumeration returns a single well-formed entry and the division is well defined. This costs nothing and is required on any Optimus-class laptop.

### E. The evidence that reversed an hour of reasoning

The log reported the graphics adapter as vendor 0x1002, device 0x73df—an AMD Radeon. On a machine with two GPUs, running a test intended to use the NVIDIA card, this reads unambiguously as “the override did not take.”

It is the opposite. **DXVK deliberately reports an AMD device while running on NVIDIA.** Two options, dxgi.nvapiHack and dxgi.hideNvidiaGpu, default to enabled and exist because a number of shipped games take a worse code path when they detect an NVIDIA adapter through DXGI. Seeing the AMD identifiers is therefore *positive evidence that the NVIDIA card is in use*.

This is worth generalising. **A compatibility layer’s job is to lie convincingly, so its self-reports are the worst available evidence about what hardware is in use.** Ground truth must come from outside the layer—in this project, from nvidia-smi --query-compute-apps, which reports the process list the kernel driver actually has.

### F. Hypotheses eliminated at this stage

TABLE XVII  
ELIMINATED DURING FIRST BRING-UP

| Hypothesis                  | How it died                                                                                         |
|-----------------------------|-----------------------------------------------------------------------------------------------------|
| Vendor-gated crash          | Spoofing the vendor as Intel (8086) and as NVIDIA (10de) both crash at the <i>identical</i> address |
| AMD helper library at fault | amd_agx_x64.dll is a hard static import; removing it yields c0000135 before any rendering           |
| Never reached rendering     | The log shows the swapchain created and presented                                                   |

The identical crash address under three different spoofed vendors is the strongest of these: a vendor-conditional code path would branch, and branching code does not fault at one address.

### G. Where this leaves the investigation

Upgrading the translation layer resolves the missing interface, and the virtual desktop resolves the display-mode fault. The game then stops crashing—and starts doing something much harder to diagnose, which is the subject of Section IX. Before that, Section VIII deals with a finding that had to be retracted, because for three sessions the discrete GPU was believed to be incapable of Vulkan at all.

## VIII. A RETRACTED FINDING: THE DISCRETE GPU WAS NEVER BROKEN

For three working sessions the recorded conclusion was that the discrete GPU could not do Vulkan on this system at all—that its driver was, in the words written down at the time, dead. That conclusion was wrong, it was wrong for a reason that generalises, and correcting it is what made everything in Sections IX through XI possible.

### A. The symptom

Any attempt to create a Vulkan instance restricted to the NVIDIA driver failed at the earliest possible point:

```
Could not get 'vkCreateInstance' via  
'vk_icdGetInstanceProcAddr' for ICD libGLX_nvidia.so.0
```

The loader could open the driver library but could not obtain a single entry point from it. Every global function resolved to null. This is not a subtle failure; it looks exactly like a broken or mismatched driver.

### B. Isolating it below the compatibility layer

The first useful decision was to stop testing through Wine. A twenty-line C program was written that does nothing but `dlopen` an ICD library, call its negotiation entry point across every interface version the loader supports, and attempt one instance creation.

TABLE XVIII

DIRECT ICD PROBE, OUTSIDE WINE AND OUTSIDE THE VULKAN LOADER

| Probe                     | Mesa pin | NVIDIA present |
|---------------------------|----------|----------------|
| icdNegotiate vl-7         | -3       | 0              |
| vkCreateInstance          | -3       | 0              |
| Resulting instance handle | (nil)    | 0x557e90579430 |

The difference between the two columns is a single environment variable. That is the whole finding: the driver is fine, and something in the environment is disabling it.

### C. Root cause: an environment variable that replaces rather than adds

The system pins its EGL vendor selection through `__EGL_VENDOR_LIBRARY_FILENAMES`, set to Mesa's `50_mesa.json` so that ordinary desktop applications do not incidentally wake the discrete GPU.

**That variable is a replacement, not an addition.** It does not extend the vendor search path; it is the vendor search path. With only Mesa's entry visible, `libglvnd` [18] has no NVIDIA vendor at all.

The non-obvious part is why this breaks *Vulkan*, which is a separate API with its own loader and its own ICD list. NVIDIA's Vulkan implementation initialises through the same vendor-dispatch machinery as its EGL implementation. With the vendor absent, that initialisation fails silently and the ICD returns a table of null pointers rather than an error.

The confirming observation is from `strace`: in the failing configuration the driver library **never opens `/dev/nvidiactl`**. It is not being refused by the kernel; it never asks. This also independently eliminates the access gate of Section II, which was again the first suspect.

### D. Fix

Whenever the NVIDIA Vulkan driver is required, the EGL vendor list must name *both* vendors—Mesa's entry and `60_nvidia.json`. The rule to carry forward:

**A pin that names one vendor is a policy statement, and every consumer of that policy inherits it, including ones in a different API.** Any launcher that needs the pinned-out vendor must restore it explicitly rather than assume it can add to a list.

After the correction, `vulkaninfo` reports the discrete GPU exactly as it should: **NVIDIA GeForce RTX 4050 Laptop GPU**, device type `DISCRETE_GPU`, Vulkan 1.4.329, driver 595.71.05.

### E. A second, independent fault hiding behind the first

Correcting the vendor list revealed that a second defect had been masked by it. The project's Wine launcher, when asked for the discrete GPU, was performing a bare `exec wine` with no ICD selection at all—so it silently used the integrated Radeon while the desktop notification it emitted said NVIDIA.

Two independent faults, one of which lies to the user about which hardware is in use, are a good illustration of why the ground-truth rule from Section VII matters. The failing configuration was indistinguishable from the working one by every self-report available, and distinguishable instantly by `nvidia-smi --query-compute-apps`.

### F. The methodological lesson

The recorded claim was "NVIDIA Vulkan is dead on this machine." The evidence for it was a test that had been run under a system-wide policy the test did not know about.

**When an instrument reports that a major component is entirely non-functional, suspect the instrument first.** Total failure is rare; misconfigured measurement is common. The cheapest discriminator is to reproduce the failure at the lowest possible layer—here, forty lines of C with no loader, no compatibility layer, and no launcher script between the test and the driver.

A corollary specific to this class of bug: a bare test of the form "set the ICD list and see whether it works" is *not* a test of the driver, because the ICD list is not the only list involved. It is a test of one variable in an environment with at least two.

With the discrete GPU restored to service, the game could finally be run on it. It then produced the failure that defines the rest of this paper.

## IX. THE CENTRAL DEFECT: NINE INVISIBLE DIALOGS

This is the failure that consumed most of the project. It is also the second independent occurrence of the failure shape first met in Section V: *a program that is not hung, waiting on a window nobody can see*.

### A. Symptom

On the discrete GPU, the game launches and appears to work. Specifically:

- A window opens, correctly sized, and stays open.
- Roughly 1,960 MiB of video memory is allocated on the discrete GPU.
- The expected worker threads exist: vkd3d-swapchain, vkd3d\_queue, vkd3d\_fence.
- Then GPU utilisation falls to 0% and the shader clock drops to its idle 210 MHz.
- The window is black. It stays black indefinitely.
- **Nothing further is written to any log.**

The process is alive, holding memory, holding a device, and doing nothing. No error, no crash, no timeout.

### B. Making the invisible visible

The breakthrough was not a hypothesis. It was asking the window manager inside the compatibility layer to enumerate its own windows:

```
winedbg --command "info wnd"
```

There were **nine** windows of class #32770—the Win32 dialog box class—each containing three buttons reading &Abort, &Retry, &Ignore. Nine assertion dialogs, stacked, none of them ever painted to the screen, each blocking a thread on a click that will never arrive.

The assertion text is:

```
../src-wine/dlls/winevulkan/loader_thunks.c:3151  
!status && "vkCreateComputePipelines"
```

1) *The detail that redirected the entire investigation:* The natural reading is that a Vulkan call returned an error, and the natural next step is to capture the `VkResult`. That is a category error, and recognising it saved an unknown amount of further wasted effort.

**status here is an NTSTATUS describing whether the call into the Unix-side driver completed, not a VkResult describing what it returned.** Its value is `0xc0000005`: an access violation. The driver did not reject the pipeline. **The driver crashed while compiling it**, and the thunk layer is reporting the crash of the call itself.

Turning on structured-exception tracing gives the address:

```
Exception 0xc0000005 at 0x7f04a4746fd8:  
libnvidia-glvkspirv.so.610.57.04 + 0x346fd8
```

Nine threads, nine faults, one address—inside NVIDIA's SPIR-V shader compiler. This is not a game bug and not a translation-layer bug in the ordinary sense. It is a driver segmentation fault, reached deterministically.

2) *A cheaper proxy for the same signal:* Attaching a debugger to every test run is slow. A far cheaper indicator turned out to be already present in the ordinary log:

```
fixme:oleacc:find_class_data unhandled window  
class: L"#32770" (x29)  
fixme:oleacc:find_class_data unhandled window  
class: L"Button" (x9)
```

The accessibility layer announces every window class it does not recognise, so counting those lines counts the dialogs without a debugger. This became the project's primary metric.

Two traps in reading the debugger output are worth recording. `winedbg` truncates window text to fourteen characters, so button labels appear cut off and look corrupted. And the game window is reported with a geometry of `960x568`—which reads as a scaling bug and is not one, for the reason given in Section II. It was also simply sitting behind a browser window.

### C. The differential that names the culprit

The same build, the same game, the same command line, on the integrated Radeon: it renders. On the discrete NVIDIA card: nine faults in the shader compiler.

One variable differs, and it is the driver. Everything after this point is about *why* the same program produces shader code that one vendor's compiler survives and the other's does not—which is answered, unexpectedly, in Section X.

### D. An automated harness, and why it had to be negative-tested

Manual testing of a failure that takes 40–90 seconds to manifest, has no error message, and must be counted rather than observed, is not viable. A harness was built:

```
007-try.sh <label> [--igpu] [--proton=NAME] \  
[--wait=N] [--kill] [VAR=VAL ...]
```

It launches one configuration, waits, samples, kills, and emits a single line:

```
alive=Y assert=9 gpu=0 rss=2107448 secs=62
```

**The harness was then run against a configuration already known to fail, to confirm it reported failure.** This is not ceremony. An earlier version of the assertion counter used a construct of the form

```
N=$(grep -c 'pattern' "$LOG" || echo 0)
```

which is wrong in a way that is invisible: `grep -c` prints 0 and *also* exits non-zero when there are no matches, so the fallback appends a second zero and `N` becomes the two-line string `"0\n0"`. Every subsequent numeric comparison then errors and evaluates false, and the guard never fires. A health check that cannot report ill health is worse than no health check, because it is trusted.

TABLE XIX

EARLY ELIMINATIONS, EACH WITH THE MEASUREMENT THAT KILLED IT

| Hypothesis                | Measurement                                                                                                                                                                |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Still compiling shaders   | The 259 MB pipeline cache’s file descriptor position reaches the file size exactly, then stops. A second run with the cache fully warm reaches the identical stall in 46 s |
| Out of video memory       | Stopping the resident language-model service freed 3,576 MiB of 6,141. Identical stall                                                                                     |
| Online check or licensing | <b>Zero</b> TCP sockets open in the process at the time of the stall                                                                                                       |
| Missing virtual desktop   | Present and correct; the display-mode fault of Section VII does not recur                                                                                                  |
| Missing Vulkan extension  | All six extensions the translation layer requests are present and reported by the driver                                                                                   |
| Discrete-GPU access gate  | Reproduces identically as root with the gate open (Section II)                                                                                                             |

### E. Hypotheses eliminated before the shader work began

The first row deserves emphasis because it is the hypothesis that survives longest by default. “It is still working” is unfalsifiable without a measurement, and the measurement here is unusually direct: the operating system will report how far into a file a process has read. When that offset equals the file size and stops advancing, reading is finished, and “still loading” is dead.

### F. A trap in process matching

The harness matches the game process by name. On Linux the kernel’s `comm` field is truncated to fifteen characters, so the process is `007FirstLight.e` and never `007FirstLight.exe`. A pattern containing the full name matches nothing, forever, silently—and the harness reports `alive=N` for a process that is running perfectly well.

The related hazard, which bit earlier in the project, is that a substring match against the full command line (`pgrep -f`) matches the shell that is performing the search. The correct tool is an exact-name match against the truncated `comm`.

## X. THE SHADERS: SAME GAME, DIFFERENT BYTES PER VENDOR

The driver crashes compiling a shader. The obvious next question is *which* shader, and the obvious assumption is that both GPUs receive the same shaders and only one vendor’s compiler mishandles them. The second half of that assumption is false, and correcting it reframes the problem.

### A. Dump both, and compare

The translation layer can be told to write every shader it produces, in both its input and output forms, to a directory. Running the game once on each GPU and comparing the two directories gives the following.

The interpretation is unambiguous. **The translation layer emits vendor-specific code.** Given identical Direct3D shader bytecode as input, it produces different Vulkan output depending on which driver is present, because it uses vendor extensions where they are available.

TABLE XX

SHADER OUTPUT COMPARED ACROSS THE TWO GPUS, SAME GAME, SAME BUILD

| Measurement                                          | Value                          |
|------------------------------------------------------|--------------------------------|
| Modules translated, Radeon                           | 6,657                          |
| Modules translated, NVIDIA                           | 4,430                          |
| Hashes present in both sets                          | 4,430                          |
| Same hash, <b>different bytes</b>                    | <b>4,047</b>                   |
| Uses                                                 | 4,039 NVIDIA / <b>0</b> Radeon |
| SPV_NV_raw_access_chains<br>of which compute shaders | 561 of 886                     |
| Uses SPV_NV_shader_<br>subgroup_partitioned          | exactly 1 compute shader       |

Of the shaders present on both sides, **91% differ byte-for-byte**. The NVIDIA path uses a vendor extension in 4,039 modules that the Radeon path uses in none. So the premise “the same shaders work on AMD” was never true. AMD was never given these shaders.

This matters for a reason beyond diagnosis: it means the AMD success provided much weaker evidence than it appeared to. It did not show that the game’s shaders are sound; it showed that a *different* set of shaders is sound.

### B. Overriding the shaders, and a large partial result

The translation layer also supports substituting a shader by hash: place a replacement in a directory and the corresponding module is loaded from there instead of being generated. Feeding the Radeon’s modules to the NVIDIA run is therefore mechanically possible.

The result was the single largest improvement of the investigation:

9 dialogs, GPU 0% → **4 dialogs, GPU 100%**

Five of the nine crashing pipelines stopped crashing, and the discrete GPU went from idle to fully loaded—it was doing real work for the first time. The result held across **fourteen consecutive invariant runs**, so it is not noise.

It is also not a fix. Four dialogs remain, and the configuration is not one that could ship: it requires a shader dump from the other GPU, and it forces the translation layer to bypass its own pipeline cache.

### C. Why this experiment is not clean, stated plainly

It is tempting to read the partial success as “the vendor extension is the bug.” The experiment does not support that, for two reasons that must be stated:

- 1) Substituting shaders **forces the layer to ignore its cached SPIR-V entirely**, changing a second variable at the same time as the first.
- 2) The Radeon modules were compiled for a different device’s capability set. Feeding them to a different vendor’s compiler is not a controlled substitution; it is a different program.

#### D. An unresolved contradiction, published rather than smoothed over

If the vendor extension were the cause, then disabling that extension outright should reproduce the improvement. It does not.

Disabling `VK_NV_raw_access_chains` through the layer’s own extension filter—so that the same 4,039 modules are generated without it—does **not** reduce the dialog count. The override, which changes both the extension usage and the cache path, does.

We were unable to resolve this contradiction, and we report it unresolved. The most likely explanations are that the improvement came from the cache bypass rather than the extension, or that the extension filter does not affect already-cached modules. Neither was confirmed. It is recorded here because a reader reproducing this work will hit the same contradiction, and because the correct fix, found later, makes the question moot in a way that is itself informative: the layer that emitted the shaders was the wrong layer to be adjusting.

#### E. Everything tried at the shader level

TABLE XXI  
SHADER- AND TRANSLATION-LAYER CONFIGURATION ATTEMPTS, ALL  
NEGATIVE

| Attempt                                    | Result                            |
|--------------------------------------------|-----------------------------------|
| Disable the vendor extension               | no change                         |
| Disable subgroup-partitioned ops           | no change                         |
| Force single-threaded pipeline compilation | no change                         |
| Disable the pipeline library               | no change                         |
| Disable pipeline caching entirely          | no change                         |
| Force synchronous shader compilation       | no change                         |
| Disable descriptor-buffer usage            | no change                         |
| Disable ray tracing                        | no change                         |
| Disable mesh shaders                       | no change                         |
| Force a conservative memory model          | no change                         |
| Reduce shader-compiler thread count        | no change                         |
| Substitute the other vendor’s modules      | <b>9 → 4 dialogs, GPU to 100%</b> |

Eleven configuration changes with no effect and one with a large effect is itself a signal. When a component’s entire exposed configuration surface fails to move the failure, the component is probably not the one to be configuring.

#### F. A verification script that was itself wrong

The override mechanism prints a line for each substituted module, so the harness checked for those lines to confirm the override had taken effect. It reported success on runs where nothing had been substituted, because of the `grep -c` defect described in Section IX. Two sweep results had to be discarded and re-run.

**Every instrument in this project that was not deliberately tested against a known-bad input**

**was, at some point, wrong.** The harness, the override verifier, the shader-count comparison, and the process matcher each produced a confident false reading before being corrected.

At this point the shader hypothesis was exhausted: a real and reproducible improvement, no clean explanation, and no path to a shippable configuration. The next section changes a variable that had been held constant since the beginning of the project, and the problem disappears entirely.

### XI. THE FIX: CHANGING THE VARIABLE NOBODY WAS VARYING

#### A. The unexamined constant

Every experiment in Sections IX and X varied something inside the translation layer or the driver, and every one of them held the *compatibility distribution* fixed. GE-Proton 10-34 had been chosen at the start because it worked, and it was never afterwards treated as a variable.

A compatibility distribution bundles its own build of the Direct3D 12 translation layer. Changing the distribution changes that layer. The question “does a different build of the translation layer produce shaders the driver survives?” had therefore never been asked, despite the entire investigation being about shaders the driver does not survive.

#### B. Building a variant cheaply

Replacing the bundled layer in place would destroy a working configuration. The approach taken instead was to clone the distribution and replace only the two libraries of interest.

```
cp -al "$SRC" "$DST" # hardlink clone
for d in <four library directories>; do
  rm -f "$DST/$d/d3d12.dll" "$DST/$d/d3d12core.dll"
  cp "$NEW/d3d12.dll" "$DST/$d/"
  cp "$NEW/d3d12core.dll" "$DST/$d/"
done
```

A hardlink clone of a 1.4GB distribution costs about **25MB**, because only the replaced files are new data. This is safe *only* because the replacement is `rm` followed by `cp`—creating a new inode—rather than an in-place edit, which would modify the original through the shared inode. That distinction is the entire safety argument and is easy to get wrong.

Two further points, each of which cost a wasted result:

- **A prefix upgrade is one-way.** Once an emulated Windows filesystem has been upgraded by a newer distribution it cannot be used by an older one. Each variant needs its own prefix.
- **Seeding a prefix requires more than the prefix directory.** Copying only `pfx/` omits the sibling `tracked_files` and `version` metadata, and the launcher aborts before the game starts—which reads exactly like the variant being broken. This defect made the *winning* configuration look like a failure on its first run.

TABLE XXII  
TRANSLATION-LAYER BUILDS, ALL OTHER VARIABLES HELD FIXED

| Build                      | Layer        | Dialogs  | GPU         |
|----------------------------|--------------|----------|-------------|
| Stock (as used throughout) | 2.14.1       | 9        | 0%          |
| Clone with newer layer     | <b>3.0.1</b> | <b>0</b> | <b>100%</b> |
| Clone with same layer      | 2.14.1       | —        | exits early |
| Newer distribution         | 3.x          | 0        | 34%         |

### C. Result

**Zero dialogs. The game runs.** The driver’s shader compiler does not crash, because it is no longer being handed the code that crashed it.

The third row is the control: the same clone procedure with the *original* layer does not fix anything, which rules out the cloning itself as the cause of the improvement.

TABLE XXIII  
BEFORE AND AFTER, DISCRETE GPU

|                   | Before              | After                     |
|-------------------|---------------------|---------------------------|
| Assertion dialogs | 9                   | <b>0</b>                  |
| GPU utilisation   | 0%                  | <b>100%</b>               |
| Video memory      | 1,960 MiB, static   | 2,064–2,070 MiB, live     |
| Shader clock      | 210 MHz idle        | boost                     |
| Window            | black, indefinitely | title screen at 2880×1800 |

### D. Verified three independent ways

Following the project rule that a fix is not a fix until it has been exercised end to end, the result was confirmed by three methods that do not share a failure mode:

- 1) A **199-second continuous run** with the kernel driver’s own compute process list sampled throughout—ground truth from outside the compatibility layer, per Section VII.
- 2) A **screenshot of the in-game graphics menu**, with DLSS listed among the upscaling options. That option is only offered when an NVIDIA adapter is present, so the game itself corroborates which GPU it is on.
- 3) A **screenshot of the title screen** taken from the installed launcher command as an ordinary user would invoke it—not from the test harness.

The third matters more than it looks. A fix that works only under the harness is a fix to the harness.

### E. The integrated GPU was re-tested, not assumed

The Radeon path was re-run under the new configuration rather than presumed intact: `assert=0`, 95% utilisation. Both GPUs work. The launcher defaults to the discrete card and accepts a flag to select the integrated one.

### F. Launcher design decisions

Three small decisions in the shipped launcher are worth recording because each encodes something learned above.

**Restore the EGL vendor list explicitly.** Per Section VIII, the system-wide pin must be widened to include the NVIDIA vendor, not assumed to be additive.

**Detect the panel’s native mode from the kernel, not from a configuration file.** A subtlety: files under `/sys` report a size of zero regardless of content, so the common shell idiom for “file exists and is non-empty” is always false there. The file must be read and its content checked.

**Report memory pressure; do not act on it.** The discrete card’s memory is a hard 6,141 MiB budget, and a resident language-model service commonly holds 3,576 MiB of it. The launcher prints a notice when free memory is low. It does **not** stop the service, because that service may be doing work the user cares about more than the game. Tools that silently kill other work to make room for themselves are a category of behaviour to avoid.

### G. Closing the remaining lead

One lead from the original three-way plan remained: try a newer graphics driver. It is closed negatively—all available prerelease driver packages resolve to the same version already installed, 610.57.04. There is no newer driver to try. The lead is exhausted rather than unexplored, which is a different and more useful state to record.

### H. Undo

The original distribution is untouched; the variant is a separate directory with its own prefix. Reverting is deleting two directories and one line in the launcher. The fix is additive, and the cost of being wrong about it is a few seconds.

## XII. DISCUSSION: WHAT GENERALISES

Nothing in this project required a novel technique. What it required, repeatedly, was noticing that a question had not been asked. This section extracts the parts that transfer.

### A. Search order dominates search effort

The defect was in a component that was never varied, and it was found within one attempt of finally varying it. The preceding weeks were spent varying components that could not have been responsible.

The failure of process is identifiable in advance. **A component chosen at the start of a project because it worked becomes invisible to the project.** It is not on the list of suspects because it is part of the definition of the setup.

The practical rule follows from delta debugging [6], applied one level up from where it is usually applied:

**Before bisecting within a component, enumerate the components you have never bisected across.**

Any element of the stack that has held one value for the whole investigation belongs on the suspect list *ahead of* the elements you have already varied without success, not behind them.

A corroborating signal was available and misread: eleven configuration changes inside the translation layer produced no effect at all (Table XXI). An exhausted configuration surface is



evidence that the component is the wrong one to configure—not an invitation to configure it harder.

### B. Differential diagnosis beats hypothesis generation

Twice in this project, two failures shared a symptom and were separated by one cheap observation:

- Extraction: process alive, nothing happening. *Is a core busy, and did it write any bytes?* Two seconds of observation, two entirely different root causes (Table IX).
- Rendering: window open, nothing happening. *Ask the window manager what windows exist.* One command, and an invisible modal dialog becomes a driver crash address.

In both cases the correct move was to *enumerate the state* rather than to generate hypotheses about it. Hypothesis generation is unbounded; state enumeration is finite and usually one command.

### C. Prefer pipelines whose stages publish their preconditions

The reassembly (Section VI) succeeded on the first attempt across ten stages and 54 GB because every stage could be checked against a manifest read *before* it ran. Five checks, five exact matches, and any error would have surfaced at its own stage.

Where a format offers this, it is the cheapest correctness guarantee available: a checksum you did not have to compute and did not have to trust yourself to choose.

### D. Instruments lie, and only negative testing catches it

Every measurement tool built in this project produced at least one confident false reading before being corrected:

- The assertion counter reported zero because `grep -c` both prints zero and fails.
- The override verifier reported success on runs that overrode nothing, for the same reason.
- The process matcher reported the game dead because `comm` truncates at fifteen characters.
- The graphics layer reported an AMD adapter while running on NVIDIA, by design.
- The launcher’s own notification claimed NVIDIA while using the integrated GPU.
- The recorded conclusion “NVIDIA Vulkan is non-functional” was an artefact of a system-wide policy the test was unaware of.

**The discipline that catches all six is the same: run every instrument against an input whose answer you already know, and specifically against a *failing* input.** A health check that has only ever been shown reporting health has not been tested.

### E. “Impossible” has exactly one legitimate use

This paper contains one genuine impossibility claim—the hypervisor-based countermeasure of Section IV-D, which fails for two independent structural reasons—and it is the only place the word is used.

Elsewhere, several things that were recorded as impossible turned out to be merely unexamined: a compiled installer script

that “cannot be read” was read by parsing the format; a driver that was “dead” was disabled by an environment variable; an archive that used “an unsupported codec” was missing a configuration file.

**Separate three claims that are easily conflated: this cannot exist; this would be difficult; I do not currently know how.** Only the first is a stopping condition, and it requires a structural argument, not a failed attempt.

### F. The strong form of a negative result

Section IV makes a claim of absence—that no commercial anti-tamper system was ever in this build—and supports it with a cryptographic identity rather than an accumulation of heuristics.

This is worth generalising because negative results about binaries are usually argued the weak way: entropy, section names, string searches, absence of known signatures. Each of those is defeasible individually, and stacking defeasible arguments does not produce an infeasible one. A validated signature over the current bytes does, because it converts “we looked and found nothing” into “the file is provably identical to the one the publisher signed, so there is nothing to find.”

The corollary is a cost argument: **the strong test was also the cheapest one.** It was performed after the weak ones, which is the wrong order.

### G. Threats to validity

- **Single system.** Every result comes from one laptop with one hybrid graphics configuration. The reassembly and forensic findings should transfer directly; the graphics findings are specific to this driver version, this translation layer, and this vendor pair.
- **The root cause is localised, not explained.** We show that one build of the translation layer produces shader code that crashes a specific driver’s compiler and that another does not. We did not identify the offending construct, and did not reduce it to a minimal reproducer. That would be the natural next step and would produce a filable driver bug.
- **The unresolved contradiction of Section X** is genuinely unresolved. The partial improvement from shader substitution has no confirmed mechanism.
- **Timing figures** are wall-clock on a loaded desktop system, not a controlled benchmark.

### H. Related work

Entropy analysis for packing detection follows Lyda and Hamrock [1]; the windowing refinement and the known false-positive behaviour are discussed at length in the packer-detection literature [2]. Authenticode structure and validation semantics are specified by Microsoft [3] over PKCS#7/CMS [4] and the PE format [5]. The bisection discipline in Section XII is delta debugging [6], and the general approach to systematic debugging follows Zeller [7]. The compatibility stack under test is Wine [8] with Proton [9],

vkD3d-proton [10], and DXVK [11], targeting Vulkan [13] and SPIR-V [12]. The reassembly toolchain uses LZ4 [14], VCDIFF [15], HDiffPatch [16], and Ogg Vorbis [17]; the installer format is Inno Setup [20] and the object-namespace behaviour is documented in Windows Internals [19]. Vendor dispatch is handled by libglvnd [18].

To our knowledge the specific combination reported here—a signature-based negative determination of anti-tamper presence, a manual reconstruction of a delta-compressed redistribution without its installer, and a cross-vendor shader divergence causing a driver-side compiler fault under translation—has not been described together in one place.

### XIII. CONCLUSION

Four questions were asked and four were answered.

**Is it hostile?** No. Six independent lines of static evidence, all negative, including a byte-for-byte comparison against an independently verified package. The most alarming artefact in the distribution—a utility that rewrites the system `hosts` file—turned out to be the strongest single piece of evidence for the package’s authenticity.

**Is commercial anti-tamper present?** No, and it never was in this build. The executable’s Authenticode digest validates over its current bytes, so the file is exactly what the publisher signed. Protection of that class lives inside the executable and cannot be removed without invalidating the signature. Three corroborating instruments agree, including a windowed entropy profile with zero windows anywhere near the threshold.

**Can it be reconstructed without its installer?** Yes—in 37 minutes, to a working 54 GB tree, with every one of ten stages verified in advance against a manifest the format publishes for free. The artefact that blocked the project for two working sessions was a three-line advertisement fetcher containing no build logic at all, and it was blocked not by a codec but by a missing configuration file at a hard-coded path.

**Will it run on both GPUs?** Yes. The integrated GPU worked early. The discrete GPU produced a failure with no error message, no crash, and no log output, which turned out to be nine unpainted modal dialogs each reporting that the vendor’s shader compiler had segmentation-faulted. The cause was neither the game, nor the driver, nor the hardware, nor the permissions gate that was suspected three separate times. It was the build of the Direct3D 12 translation layer—the one component chosen at the start of the project and never once treated as a variable. Replacing it took one attempt.

The recurring lesson is not about graphics. It is that in a stack of a dozen substitutable layers, the expensive mistake is rarely a wrong hypothesis; it is a component that was never a hypothesis. Twice in this project—the installer helper that re-extracted itself, and the translation layer that was assumed fixed—the answer was in a place that had been silently excluded from the search space at the outset.

Two smaller results are worth carrying away independently. A validated code signature is a stronger, cheaper, and more honest way to establish the absence of anti-tamper protection than any heuristic stack, and it should be the first test rather

than the last. And a pipeline stage that publishes its own preconditions gives you a correctness check you neither had to design nor have to trust yourself to have designed correctly.

## APPENDIX: REPRODUCTION

### A. Establishing anti-tamper absence on an arbitrary binary

- 1) Extract the certificate directory from the PE file, parse the PKCS#7 structure, and read the embedded digest.
- 2) Recompute the Authenticode hash over the file as it exists, excluding the checksum field, the certificate table entry, and the certificate data. Compare. If they match, stop—the question is answered.
- 3) Otherwise, profile entropy in windows of 1 MiB over the code section. Report the window count above 7.5 and state the threshold used. Never report a whole-file figure alone.
- 4) Enumerate sections and the import table. A full, compiler-shaped import table is difficult for a protection layer to preserve.
- 5) Identify what protection *is* present. “No anti-tamper” is not “no DRM.”

### B. Recovering a compiled Inno Setup script

Search for the 12-byte identifier `rDlPtS`; read nine little-endian 32-bit words; seek to `offset0`; skip the 64-byte banner; read CRC, stored size, and compression flag; then read `[crc32(4) + up to 4096 bytes]` chunks until the stored size is satisfied. The concatenated result is raw LZMA1 whose first five bytes are the properties block. Do *not* follow the `zlb\x1a` magic near the start of the overlay; that is the file data.

### C. Reassembling the distribution

- 1) Copy the decompressor’s configuration file to the root of the emulated system drive.
- 2) Patch the compression helpers to drop the `Global\` namespace prefix, using a live patcher that polls for newly extracted copies.
- 3) Extract all six containers to their *correct* destinations (Table XIV); two of the six do not go where the others do.
- 4) Read the worklists from the smallest container for the commands, and the carved installer script for the ordering and working directories.
- 5) Before each directory patch, read its manifest and record the expected input count, input byte total, and output size. Verify after.
- 6) Parallelise the per-file worklists; the cost is process startup, not compression.

### D. Bisecting a translation-layer failure

- 1) Enumerate windows inside the compatibility layer before assuming a hang. A modal dialog that is never painted is indistinguishable from a deadlock by every other means.

- 2) Read assertion text precisely. An NTSTATUS is not an API return code, and the difference decides whether you are debugging a rejection or a crash.
- 3) Dump shaders on both vendors and compare hashes *and* bytes. Identical hashes with differing bytes means the layer is emitting vendor-specific code and your cross-vendor control is not a control.
- 4) Vary the translation-layer build before exhausting its configuration surface, not after.
- 5) Clone the distribution with hardlinks and replace libraries by delete-then-copy. Never edit in place. Give each variant its own prefix, and copy the prefix's sibling metadata, not just the prefix.

#### E. Catalogue of silent failures encountered

Each of the following reported success, or reported nothing, while doing something other than what was intended.

TABLE XXIV  
SILENT FAILURES ENCOUNTERED IN THIS PROJECT

| Construct                                    | What it actually does                                                                                                 |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>grep -c</code> with a fallback         | prints 0 <i>and</i> exits non-zero, so the fallback appends a second zero; all later comparisons error and read false |
| Process match on full name                   | <code>comm</code> truncates to 15 characters; the pattern never matches and the process reads as dead                 |
| Substring match on command line              | matches the searching shell itself                                                                                    |
| Static patch of installer helpers            | bypassed; the installer re-extracts fresh copies at runtime                                                           |
| Prefix seeded from <code>pfx/</code> only    | missing sibling metadata aborts the launcher before the game starts                                                   |
| Library search under <code>lib64</code>      | the libraries are under <code>lib</code> ; the search returns a confident false negative                              |
| GUID searched in printed order               | the first three fields byte-swap in memory                                                                            |
| Emptiness test on a <code>/sys</code> file   | always false; <code>sysfs</code> files report zero size regardless of content                                         |
| Graphics layer's adapter report              | deliberately reports AMD while on NVIDIA                                                                              |
| Launcher's own notification                  | claimed NVIDIA while running on the integrated GPU                                                                    |
| EGL vendor pin                               | replaces the vendor list rather than extending it, disabling an unrelated API for an unrelated vendor                 |
| Setgid helper raising only the effective GID | the shell drops it at startup                                                                                         |
| Archive tool's size counter                  | 32-bit signed; goes negative past 2 GB                                                                                |
| Debugger window text                         | truncated to 14 characters; labels look corrupted                                                                     |
| Compositor geometry versus screenshots       | two different coordinate systems differing by the scale factor                                                        |

#### F. Artefacts

The harness scripts (launcher, single-configuration trial, distribution variant builder, three configuration sweeps, and a shader bisector), the archive extraction harness and its freestanding 32-bit build recipe, the installer-script carver, the helper patcher, and the reassembly drivers for both data chunks are retained alongside a running findings log. Defect and decision records are maintained in the project's own issue and decision registers.

#### REFERENCES

- [1] R. Lyda and J. Hamrock, "Using entropy analysis to find encrypted and packed malware," *IEEE Security & Privacy*, vol. 5, no. 2, pp. 40–45, 2007.
- [2] X. Ugarte-Pedrero, D. Balzarotti, I. Santos, and P. G. Bringas, "SoK: Deep packer inspection: A longitudinal study of the complexity of run-time packers," in *Proc. IEEE Symp. Security and Privacy*, 2015, pp. 659–673.
- [3] Microsoft Corporation, "Windows Authenticode portable executable signature format," Technical Specification, 2008.
- [4] R. Housley, "Cryptographic Message Syntax (CMS)," RFC 5652, IETF, 2009.
- [5] Microsoft Corporation, "PE format specification," Microsoft Learn, 2024.
- [6] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [7] A. Zeller, *Why Programs Fail: A Guide to Systematic Debugging*, 2nd ed. Morgan Kaufmann, 2009.
- [8] The Wine Project, "Wine: a compatibility layer for running Windows applications on POSIX-compliant operating systems," <https://www.winehq.org>.
- [9] Valve Corporation, "Proton: compatibility tool for Steam Play," <https://github.com/ValveSoftware/Proton>.
- [10] H.-K. Nienhuys *et al.*, "vkd3d-proton: Direct3D 12 to Vulkan translation layer," <https://github.com/HansKristian-Work/vkd3d-proton>.
- [11] P. Möller, "DXVK: Vulkan-based implementation of D3D9, D3D10 and D3D11," <https://github.com/doitsujin/dxvk>.
- [12] The Khronos Group, "SPIR-V specification, version 1.6," 2023.
- [13] The Khronos Group, "Vulkan 1.4 specification," 2024.
- [14] Y. Collet, "LZ4: extremely fast compression algorithm," <https://github.com/lz4/lz4>.
- [15] D. Korn, J. MacDonald, J. Mogul, and K. Vo, "The VCDIFF generic differencing and compression data format," RFC 3284, IETF, 2002.
- [16] Housisong, "HDiffPatch: a C++ library and command-line tool for diff and patch of binary files and directories," <https://github.com/sisong/HDiffPatch>.
- [17] Xiph.Org Foundation, "Vorbis I specification," 2020.
- [18] NVIDIA Corporation and contributors, "libglvnd: the GL vendor-neutral dispatch library," <https://gitlab.freedesktop.org/glvnd/libglvnd>.
- [19] M. Russinovich, D. Solomon, and A. Ionescu, *Windows Internals, Part 2*, 6th ed. Microsoft Press, 2012.
- [20] J. Russell, "Inno Setup," <https://jrsoftware.org/isinfo.php>.